

Анализ пользовательской активности и успешности на образовательной платформе

Бизнес-контекст: анализируем данные образовательной платформы для аналитиков. На платформе пользователи решают задачи, проходят тесты, получают очки опыта (score) и взаимодействуют с системой.

Цель анализа: Определить ключевые факторы, влияющие на:

Активность пользователей

Успешность решения задач

Удержание пользователей

```
In [1]: #Загрузка библиотек
import pandas as pd
import psycopg2
import os
from scipy.stats import *
```

```
In [2]: # Подключение к базе данных
host = '95.163.241.236'
port = '5432'
database = 'simulative'
user = 'student'
password = 'qweasd963'

tables = [
    'users',
    'userentry',
    'page',
    'company',
    'problem',
    'language',
    'coderun',
    'codesubmit',
    'test',
    'testresult',
```

```

        'transaction',
        'transactiontype'
    ]

dataframes = {}

for table in tables:
    try:
        # Открываем новое соединение для каждой таблицы
        conn = psycopg2.connect(
            host=host,
            port=port,
            database=database,
            user=user,
            password=password
        )

        df = pd.read_sql_query(f'SELECT * FROM "{table}"', conn)
        dataframes[table] = df
        print(f"{table}: {len(df)} записей")

        conn.close()

    except Exception as e:
        print(f"Ошибка при загрузке {table}: {e}")

print("Загрузка завершена!")

```

C:\Users\kozya\AppData\Local\Temp\ipykernel_26996\3085705717.py:36: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.

```
df = pd.read_sql_query(f'SELECT * FROM "{table}"', conn)
```

users: 2774 записей
userentry: 7995 записей
page: 318 записей
company: 10 записей
problem: 248 записей
language: 6 записей
coderun: 55722 записей
codesubmit: 23768 записей
test: 24 записей
testresult: 15902 записей
transaction: 21103 записей
transactiontype: 30 записей
Загрузка завершена!

```
In [3]: # Сохранение данных в CSV
os.makedirs('data', exist_ok=True)

for name, df in dataframes.items():
    df.to_csv(f'data/{name}.csv', index=False)
    print(f"{name}.csv сохранён")
```

users.csv сохранён
userentry.csv сохранён
page.csv сохранён
company.csv сохранён
problem.csv сохранён
language.csv сохранён
coderun.csv сохранён
codesubmit.csv сохранён
test.csv сохранён
testresult.csv сохранён
transaction.csv сохранён
transactiontype.csv сохранён

```
In [4]: # Загрузка нужных таблиц
users = pd.read_csv('data/users.csv')
userentry = pd.read_csv('data/userentry.csv')
page = pd.read_csv('data/page.csv')
company = pd.read_csv('data/company.csv')
problem = pd.read_csv('data/problem.csv')
coderun = pd.read_csv('data/coderun.csv')
```

```

codesubmit = pd.read_csv('data/codesubmit.csv')
test = pd.read_csv('data/test.csv')
testresult = pd.read_csv('data/testresult.csv')
transaction = pd.read_csv('data/transaction.csv')
transactiontype = pd.read_csv('data/transactiontype.csv')

```

```

In [5]: # Функция для проверки нормальности
def is_normal(series, alpha=0.05, name=''):
    data = series.dropna() #удаление пустых значений
    if len(data) < 3:
        return 'недостаточно данных'
    stat, p = shapiro(data)
    verdict = 'нормальное' if p>=alpha else 'не нормальное'
    print(f'{name}: p={p:.6f} {verdict}')
    return verdict

```

Гипотеза 1 - Среднее количество очков у пользователей из разных компаний различается

H0 - нет значимой разницы

H1 - есть значимая разница

Обоснование гипотезы: В разных компаниях могут быть разные требования, мотивация, уровень подготовки сотрудников

```

In [6]: print(f"users: {len(users)} записей")
print(f"codesubmit: {len(codesubmit)} записей")
print(f"userentry: {len(userentry)} записей\n")

```

```

users: 2774 записей
codesubmit: 23768 записей
userentry: 7995 записей

```

```

In [7]: #Просмотр уникальных компаний в таблице users
unique_companies = users['company_id'].unique()
print("Уникальные компании")
print(f"Всего групп: {len(unique_companies)}")
print(f"ID: {sorted(unique_companies, key=lambda x: 0 if pd.isna(x) else x)}")
print()

```

```
#Просмотр количества пользователей по группам
company_counts = users['company_id'].value_counts(dropna=False).reset_index()
company_counts.columns = ['company_id', 'user_count']

#Замена NaN на 'Без компании' для красивого вывода
company_counts['company_label'] = company_counts['company_id'].apply(
    lambda x: 'Без компании' if pd.isna(x) else f'Компания {int(x)}'
)

print("Количество пользователей по группам")
print(company_counts[['company_label', 'user_count']].to_string(index=False))
print()

#Группы с достаточным количеством (> 5)
large_groups = company_counts[company_counts['user_count'] >= 5]
print(f"Группы с >= 5 пользователями ({len(large_groups)} шт.)")
print(large_groups[['company_label', 'user_count']].to_string(index=False))
```

Уникальные компании

Всего групп: 8

ID: [np.float64(nan), np.float64(1.0), np.float64(2.0), np.float64(4.0), np.float64(6.0), np.float64(7.0), np.float64(8.0), np.float64(10.0)]

Количество пользователей по группам

company_label	user_count
Без компании	2530
Компания 1	133
Компания 4	47
Компания 7	40
Компания 10	12
Компания 6	10
Компания 2	1
Компания 8	1

Группы с >= 5 пользователями (6 шт.)

company_label	user_count
Без компании	2530
Компания 1	133
Компания 4	47
Компания 7	40
Компания 10	12
Компания 6	10

```
In [8]: users['company_id_clean'] = users['company_id'].fillna(-1)

#Сбор групп
groups = []
labels = []
for cid in sorted(users['company_id_clean'].unique()):
    data = users[users['company_id_clean'] == cid]['score'].dropna()
    if len(data) >= 5:
        groups.append(data)
        labels.append('Без компании' if cid == -1 else f'Компания {int(cid)}')

print("Группы")
for lab, g in zip(labels, groups):
    print(f" {lab}: {len(g)} чел.")
```

Группы

Без компании: 2530 чел.
Компания 1: 133 чел.
Компания 4: 47 чел.
Компания 6: 10 чел.
Компания 7: 40 чел.
Компания 10: 12 чел.

```
In [9]: print("Нормальность")
normal = True
for lab, g in zip(labels, groups):
    _, p = shapiro(g)
    print(f" {lab}: p = {p:.6f} → {'нормальное' if p >= 0.05 else 'не нормальное'}")
    if p < 0.05:
        normal = False
```

Нормальность

Без компании: $p = 0.000000 \rightarrow$ не нормальное
Компания 1: $p = 0.000000 \rightarrow$ не нормальное
Компания 4: $p = 0.000000 \rightarrow$ не нормальное
Компания 6: $p = 0.000017 \rightarrow$ не нормальное
Компания 7: $p = 0.000000 \rightarrow$ не нормальное
Компания 10: $p = 0.000003 \rightarrow$ не нормальное

Вывод: Так как все группы не нормальные ($p < 0.05$) я буду использовать непараметрический тест Kruskal-Wallis

```
In [10]: stat, p = kruskal(*groups)
print(f" p-value = {p:.6f}")

if p < 0.05:
    print("Разница есть: компании отличаются по среднему score")
else:
    print("Разницы нет: компании не отличаются по среднему score")
```

p-value = 0.000000

Разница есть: компании отличаются по среднему score

Итог первой гипотезы: Компании отличаются по среднему количеству очков опыта

Это указывает на то, что компании по-разному вовлекают своих сотрудников в обучение на платформе. В компаниях с высоким score пользователи активнее решают задачи и накапливают больше очков опыта, что может быть связано с корпоративной культурой,

наличием обязательных программ обучения или системой мотивации.

Рекомендация: Провести дополнительный анализ компаний с низким score, чтобы понять причины их низкой вовлечённости, и предложить им индивидуальные решения (например, введение бонусов за обучение или адаптация контента под уровень подготовки сотрудников).

Гипотеза 2. Успешность решений зависит от языка программирования

Обоснование: некоторые языки сложнее для изучения (Python сложнее, чем SQL)

```
In [11]: import pandas as pd
from scipy.stats import chi2_contingency

# Загружаем данные
codesubmit = pd.read_csv('data/codesubmit.csv')
language = pd.read_csv('data/language.csv')

# Проверяем, есть ли таблица Language с названиями
if 'name' in language.columns:
    # Объединяем, чтобы видеть названия языков
    codesubmit_with_lang = pd.merge(codesubmit, language, left_on='language_id', right_on='id', how='left')
    lang_counts = codesubmit_with_lang.groupby(['language_id', 'name']).size().reset_index(name='count')
    print(lang_counts[['language_id', 'name', 'count']].to_string(index=False))
else:
    lang_counts = codesubmit['language_id'].value_counts().reset_index()
    lang_counts.columns = ['language_id', 'count']
    print(lang_counts.to_string(index=False))

print()

print('Таблица сопряженности: язык * успех/ошибка')
#Таблица сопряженности: язык * успех/ошибка
contingency = pd.crosstab(codesubmit['language_id'], codesubmit['is_false'])

# Добавляем названия языков
if 'name' in language.columns:
    contingency.index = contingency.index.map(
```

```

        lambda x: language[language['id'] == x]['name'].values[0] if x in language['id'].values else str(x)
    )
print(contingency)
print()

#Тест хи-квадрат
chi2, p, dof, expected = chi2_contingency(contingency)

if p < 0.05:
    print("Вывод: Отвергаем H0")
    print("Успешность решений ЗАВИСИТ от языка программирования")
    print("Есть языки, на которых пользователи ошибаются чаще или реже")
else:
    print("Вывод: Не отвергаем H0")
    print("Нет связи между языком программирования и успешностью")
    print("Все языки примерно одинаковы по успешности")

```

```

language_id  name  count
          2   SQL  12368
          3 Python 11400

```

Таблица сопряженности: язык * успех/ошибка

```

is_false      0      1
language_id
SQL           5043  7325
Python        4087  7313

```

Вывод: Отвергаем H₀

Успешность решений ЗАВИСИТ от языка программирования

Есть языки, на которых пользователи ошибаются чаще или реже

Вывод 2 гипотезы: Успешность решений зависит от языка программирования

Это подтверждает, что разные языки имеют разную степень сложности для пользователей. В частности, языки с более сложным синтаксисом (Python) могут требовать большего опыта и дополнительных учебных материалов. В то же время языки с низким порогом входа (SQL) демонстрируют более высокую долю успешных решений.

Гипотеза 3. Компании различаются по активности пользователей.

Обоснование: Корпоративная культура, наличие обязательных заданий, бонусы

```
In [12]: users = pd.read_csv('data/users.csv')
users
```

Out[12]:

	id	username	first_name	last_name	is_active	date_joined	email	referral_user	company_id	tier
0	198	Pbnnd	NaN	NaN	1	2021-11-23 13:58:52.051036	jbsnjbf@gmail.com	NaN	NaN	1
1	52	fjbis	Элина	Полякова	1	2021-09-01 12:04:31.148715	fjbis@yandex.ru	NaN	NaN	1
2	331	jjxbfox	Эмилия	Афанасьев	1	2021-12-12 07:03:00.583264	jjxbfox@bk.ru	NaN	NaN	1
3	416	nsosabisfsobm	NaN	NaN	1	2022-01-02 12:22:43.014039	isfsobm.nseo@gmail.com	NaN	1.0	1
4	19	Fofbx196	Эмилия	Фролова	1	2021-04-22 15:37:33.000000	koonbnanojj.jofbx@yandex.ru	NaN	NaN	1
...	...	...	...	...	...	...	...	...	...	...
2769	1706	Domsd	Константин	Хохлова	1	2022-02-18 05:21:31.631400	abd-mbmb@ya.ru	NaN	NaN	1
2770	2574	odbfonn	Яна	Сухарева	1	2022-04-01 12:31:14.559731	nbjsbjds@mail.ru	NaN	4.0	1
2771	2660	cookboo	Артём	Хохлова	1	2022-04-08 21:01:54.011350	cookboo@gmail.com	NaN	NaN	1
2772	2816	dbkbxdaasna	NaN	NaN	1	2022-04-21 10:58:08.609494	fdsnbfo@example.org	NaN	4.0	1
2773	2817	jbesnab_ooojo	NaN	NaN	0	2022-04-21 12:20:00.000202	enbfseo71@example.net	NaN	NaN	0

2774 rows × 11 columns



```
In [13]: userentry = pd.read_csv('data/userentry.csv')
userentry
```

Out[13]:

	id	entry_at	page_id	user_id
0	1	2021-04-13 08:31:45.000000	4	1
1	2	2021-04-11 16:04:20.000000	5	8
2	3	2021-04-24 07:29:36.000000	9	4
3	4	2021-04-26 20:36:03.000000	2	13
4	5	2021-04-27 16:20:12.000000	5	14
...	...	...	...	...
7990	7991	2022-05-24 20:25:00.557579	4	3261
7991	7992	2022-05-24 20:22:12.811344	4	3293
7992	7993	2022-05-24 20:30:49.899845	4	3294
7993	7994	2022-05-24 21:11:10.407605	4	2961
7994	7995	2022-05-24 22:00:12.178406	4	121

7995 rows × 4 columns

In [14]:

```
#Подсчет количества входов для каждого пользователя
entries_count = userentry.groupby('user_id').size().reset_index(name='entry_count')

# Объединение с таблицей users
merged = pd.merge(users, entries_count, left_on='id', right_on='user_id', how='left').fillna(0)

print(f"Всего пользователей: {len(merged)}")
print(f"Среднее количество входов: {merged['entry_count'].mean():.2f}")
print(f"Максимум входов: {merged['entry_count'].max()}")
```

Всего пользователей: 2774

Среднее количество входов: 2.55

Максимум входов: 142.0

In [15]:

```
groups = []
labels = []
```

```

for cid in sorted(merged['company_id'].unique()):
    # Заменяем NaN на -1 для удобства
    cid_clean = -1 if pd.isna(cid) else cid

    # Берем данные по этой компании
    group_data = merged[merged['company_id'] == cid]['entry_count'].values

    # Оставляем только группы с >= 5 пользователями
    if len(group_data) >= 5:
        groups.append(group_data)
        labels.append('Без компании' if pd.isna(cid) else f'Компания {int(cid)}')

print("Группы для анализа")
for lab, g in zip(labels, groups):
    print(f" {lab}: {len(g)} пользователей, средняя активность = {g.mean():.2f}")

```

Группы для анализа

Компания 0: 2530 пользователей, средняя активность = 2.01
 Компания 1: 133 пользователей, средняя активность = 9.52
 Компания 4: 47 пользователей, средняя активность = 1.96
 Компания 6: 10 пользователей, средняя активность = 7.30
 Компания 7: 40 пользователей, средняя активность = 4.33
 Компания 10: 12 пользователей, средняя активность = 31.33

In [16]: *# Проверка нормальности*

```

normal = True

for lab, g in zip(labels, groups):
    if len(g) >= 3:
        _, p = shapiro(g)
        verdict = 'нормальное' if p >= 0.05 else 'не нормальное'
        print(f" {lab}: p = {p:.6f} → {verdict}")
        if p < 0.05:
            normal = False
    else:
        print(f" {lab}: слишком мало данных для проверки нормальности")

```

Компания 0: $p = 0.000000 \rightarrow$ не нормальное
Компания 1: $p = 0.000000 \rightarrow$ не нормальное
Компания 4: $p = 0.000000 \rightarrow$ не нормальное
Компания 6: $p = 0.001430 \rightarrow$ не нормальное
Компания 7: $p = 0.000000 \rightarrow$ не нормальное
Компания 10: $p = 0.000471 \rightarrow$ не нормальное

```
In [17]: stat, p = kruskal(*groups)

if p < 0.05:
    print("Разница есть: компании отличаются по активности")
else:
    print("Разницы нет: компании не отличаются по активности")
```

Разница есть: компании отличаются по активности

Вывод 3 гипотезы: Разница есть: компании отличаются по активности

Разница в активности является критической: пользователи из некоторых компаний заходят на платформу в десятки раз чаще, чем пользователи из других компаний. Это указывает на то, что уровень вовлечённости сотрудников напрямую зависит от корпоративных практик и политики компании.

Ключевые выводы по компаниям:

Компания 10 (средняя активность 31.33 входа) — явный лидер. Рекомендуется изучить их практики и предложить их другим клиентам.

Компания 0 (средняя активность 2.01 входа) — самый массовый клиент с низкой активностью. Это зона роста: даже небольшое увеличение активности принесёт значительный прирост вовлечённости.

Компания 4 (средняя активность 1.96 входа) — аутсайдер. Требуется отдельное внимание, чтобы понять причины низкой активности.

Гипотеза 4. Доля активных пользователей > 50%

Обоснование: Критическая метрика для оценки удержания платформы

```
In [18]: users = pd.read_csv('data/users.csv')
users
```

Out[18]:

	id	username	first_name	last_name	is_active	date_joined	email	referral_user	company_id	tier
0	198	Pbnnd	NaN	NaN	1	2021-11-23 13:58:52.051036	jbsnjbf@gmail.com	NaN	NaN	1
1	52	fjbis	Элина	Полякова	1	2021-09-01 12:04:31.148715	fjbis@yandex.ru	NaN	NaN	1
2	331	jjxbfox	Эмилия	Афанасьев	1	2021-12-12 07:03:00.583264	jjxbfox@bk.ru	NaN	NaN	1
3	416	nsosabisfsobm	NaN	NaN	1	2022-01-02 12:22:43.014039	isfsobm.nseo@gmail.com	NaN	1.0	1
4	19	Fofbx196	Эмилия	Фролова	1	2021-04-22 15:37:33.000000	koonbnanojj.jofbx@yandex.ru	NaN	NaN	1
...	...	...	...	...	...	...	...	...	...	...
2769	1706	Domsd	Константин	Хохлова	1	2022-02-18 05:21:31.631400	abd-mbmb@ya.ru	NaN	NaN	1
2770	2574	odbfonn	Яна	Сухарева	1	2022-04-01 12:31:14.559731	nbjsbjds@mail.ru	NaN	4.0	1
2771	2660	cookboo	Артём	Хохлова	1	2022-04-08 21:01:54.011350	cookboo@gmail.com	NaN	NaN	1
2772	2816	dbkbxdsaasna	NaN	NaN	1	2022-04-21 10:58:08.609494	fdsnbfo@example.org	NaN	4.0	1
2773	2817	jbesnab_ooojo	NaN	NaN	0	2022-04-21 12:20:00.000202	enbfxseo71@example.net	NaN	NaN	0

2774 rows × 11 columns



In [19]:

```
#Подсчет количества активных
active_count = users['is_active'].sum()
total_users = len(users)
active_share = active_count / total_users
```

```
print(f"Всего пользователей: {total_users}")
print(f"Активных: {active_count}")
print(f"Доля активных: {active_share:.2%}")
```

Всего пользователей: 2774

Активных: 2472

Доля активных: 89.11%

```
In [20]: # Биноминальный тест
result = binomtest(active_count, total_users, p=0.5, alternative='greater')
p = result.pvalue

if p < 0.05:
    print(f"Вывод: Отвергаем H0")
    print(f"Активных пользователей ЗНАЧИТЕЛЬНО больше 50% ({active_share:.2%})")
else:
    print(f"Вывод: Не отвергаем H0")
    print(f"Доля активных НЕ превышает 50% ({active_share:.2%})")
```

Вывод: Отвергаем H₀

Активных пользователей ЗНАЧИТЕЛЬНО больше 50% (89.11%)

Вывод 4 гипотезы: Активных пользователей ЗНАЧИТЕЛЬНО больше 50% (89.11%)

Ключевые выводы:

89.11% активных пользователей — это высокий показатель для образовательной платформы.

Только 10.89% пользователей неактивны, что говорит о низком уровне оттока.

Высокая активность пользователей создаёт прочную основу для монетизации и расширения функционала платформы.

Гипотеза 5. Доля успешных решений < 50%

Обоснование: Добавить подсказки, упростить начальные задачи

```
In [21]: codesubmit = pd.read_csv('data/codesubmit.csv')
codesubmit
```

Out[21]:

	id	created_at	code	problem_id	user_id	is_false	time_spent	language_id
0	1	2021-04-17 17:18:06.000000	Molestias quia saepe dolore asperiores est vol...	30	20	1	NaN	3
1	2	2021-04-04 06:13:37.000000	Delectus sunt quam et. Et et eos laboriosam ni...	9	8	1	NaN	3
2	3	2021-04-07 18:29:31.000000	Sint doloremque unde eveniet quia animi. Saepe...	1	1	1	NaN	3
3	4	2021-03-29 01:38:18.000000	Veniam et amet temporibus nihil et ea qui. Dol...	11	20	1	NaN	3
4	5	2021-04-13 17:44:10.000000	Distinctio est quos commodi. Exercitationem se...	13	12	1	NaN	3
...	...	...	...	...	...	...	...	...
23763	23764	2022-05-17 09:15:17.398235	with tt as (select to_char(ord_datetipe, 'YYYY...	81	171	1	0.165092	2
23764	23765	2022-05-17 09:15:35.225089	with tt as (select to_char(ord_datetipe, 'YYYY...	81	171	1	0.175682	2
23765	23766	2022-05-17 09:17:16.239885	with tt as (select EXTRACT(YEAR FROM ord_datet...	81	171	0	0.203028	2
23766	23767	2022-05-17 09:29:24.544638	with tt as (select to_char(ord_datetipe, 'DD- M...	80	171	1	0.167917	2
23767	23768	2022-05-17 09:29:58.442284	with tt as (select to_char(ord_datetipe, 'DD- M...	80	171	0	0.188663	2

23768 rows × 8 columns

In [22]:

```
#Подсчет успешных решений
# Успех = is_false = 0
successful = codesubmit[codesubmit['is_false'] == 0].shape[0]
total = len(codesubmit)
```

```
success_share = successful / total

print(f"Всего попыток: {total}")
print(f"Успешных (is_false == 0): {successful}")
print(f"Неуспешных (is_false == 1): {total - successful}")
print(f"Доля успешных: {success_share:.2%}")
```

Всего попыток: 23768
Успешных (is_false == 0): 9130
Неуспешных (is_false == 1): 14638
Доля успешных: 38.41%

```
In [23]: # Биномиальный тест
result = binomtest(successful, total, p=0.5, alternative='greater')
p = result.pvalue

if p < 0.05:
    print("Вывод: Отвергаем H0")
    print(f"Успешных решений ЗНАЧИТЕЛЬНО больше 50% ({success_share:.2%})")
else:
    print("Вывод: Не отвергаем H0")
    print(f"Доля успешных НЕ превышает 50% ({success_share:.2%})")
```

Вывод: Не отвергаем H₀
Доля успешных НЕ превышает 50% (38.41%)

Вывод 5 гипотезы: Доля успешных решений НЕ превышает 50%

Это тревожный сигнал для платформы: пользователи чаще ошибаются, чем решают задачи правильно. Такая ситуация может негативно влиять на мотивацию пользователей, их вовлечённость и желание продолжать обучение на платформе.

Ключевые выводы:

Доля успешных решений ниже 50% - задачи слишком сложные для целевой аудитории.

Пользователи теряют уверенность, когда часто ошибаются - снижается удержание.

Это влияет на ключевые метрики продукта: активность, удержание, NPS.

Возможные причины:

Задачи не соответствуют уровню подготовки пользователей.

Не хватает обучающих материалов и подсказок.

На некоторых языках программирования задачи сложнее (см. Гипотезу 2).

Пользователи не получают обратную связь при ошибках.